

ETIQUETADOR

Informe del proyecto

Desarrollo y análisis de eficiencia

Índice

1. Introducción	3
2. Descripción del proyecto	4
Clasificación de prendas con KNN	4
Detección de colores con K-Means	4
Búsqueda de productos	5
3. Desarrollo	5
4. Análisis de parámetros	6
4.1 Valor de K en KNN	6
4.2 Número de colores en K-Means	7
4.3 Inicialización de centroides	7
4.4 Características usadas en KNN	8
5. Análisis de eficiencia	8
5.1 Eficiencia de KNN	8
5.2 Eficiencia de K-Means	9
5.3 Eficiencia de find_bestK	9
5.4 Resumen de eficiencia	9
6. Conclusiones	10

1. Introducción

En este proyecto se ha creado un pequeño sistema capaz de etiquetar imágenes de productos de ropa. La idea principal es que el programa pueda decir dos cosas: qué tipo de prenda aparece y cuál es su color principal.

Esto puede servir para una tienda online, ya que permitiría hacer búsquedas sencillas como “red shirt”, “black sandals” o “pink dress”. Así, el usuario no tendría que revisar todos los productos manualmente.

Para resolver el problema se han utilizado dos algoritmos clásicos: KNN para reconocer la forma de la prenda y K-Means para detectar los colores dominantes. Aunque no son algoritmos muy complejos, son útiles para entender cómo funciona un sistema básico de clasificación y etiquetado de imágenes.

2. Descripción del proyecto

El proyecto se divide en tres partes principales: clasificación de la forma, detección del color y búsqueda de productos usando esas etiquetas.

Clasificación de prendas con KNN

1. Se suben las imágenes de entrenamiento y de prueba. (train y test)
2. Cada imagen se convierte a escala de grises para ver más la forma que en el color.
3. La imagen se transforma en un vector de píxeles. Como las imágenes son de 60 x 80, cada una queda representada por 4800 valores.
4. El algoritmo KNN compara la imagen nueva con las imágenes de entrenamiento.
5. Finalmente, asigna la clase de la prenda según los centroides más cercanos.

Las clases posibles son prendas como (dresses, shirts, jeans, shorts, sandals, flip flops, socks y handbags).

Detección de colores con K-Means

1. Se toman todos los píxeles de la imagen en formato RGB.
2. K-Means agrupa esos píxeles en varios grupos de colores.
3. Cada grupo tiene un centroide, que representa un color dominante.
4. Después, ese color RGB se traduce a un nombre de color como red, blue, black, white, etc.

Con esto se consigue que una imagen no se describa con miles de píxeles, sino con unos pocos colores importantes.

Búsqueda de productos

Una vez que cada imagen tiene una etiqueta de forma y una etiqueta de color, se puede hacer una búsqueda combinada. Por ejemplo, si el usuario escribe “pink dress”, el sistema busca imágenes que tengan la forma “dress” y alguno de los colores detectados como “pink”.

3. Desarrollo

Durante el desarrollo se han implementado varias partes del sistema. El código está separado en diferentes archivos para que sea más fácil de entender y modificar.

Archivo	Función principal
KNN.py	Implementa el algoritmo KNN y la predicción de la forma de la prenda.
Kmeans.py	Implementa K-Means para agrupar píxeles y detectar colores dominantes.
utils.py	Incluye funciones auxiliares, como convertir RGB a gris y traducir colores a probabilidades.
utils_data.py	Carga las imágenes y sus etiquetas desde el dataset.
app.py	Permite usar el sistema desde una aplicación web sencilla.
analysis.py	Prueba distintos parámetros y compara resultados.

En general, el funcionamiento del programa es bastante directo: primero se preparan los datos, luego se entrenan o configuran los algoritmos, y finalmente se usan para predecir las etiquetas de nuevas imágenes.

4. Análisis de parámetros

En este apartado se comentan algunos parámetros importantes que influyen en el resultado final. No se trata solo de que el programa funcione, sino de entender qué decisiones pueden mejorar o empeorar el sistema.

4.1 Valor de K en KNN

En KNN, la K indica cuántos vecinos se usan para decidir la clase de una imagen.

- Si K es muy pequeña, el sistema se fija mucho en la imagen más parecida. Puede ir bien si el dataset está limpio, pero también puede ser sensible al ruido.

- Si K es más grande, la decisión es más estable, pero puede mezclar prendas parecidas y perder precisión.

Por eso se prueban varios valores de K. En este tipo de imágenes, valores pequeños suelen funcionar bien porque las prendas tienen fondos simples y están bastante centradas.

- En la aplicación final (app.py) se usa K = 5 para clasificar la forma de la prenda:

```
KNN = KNN(train_gray, train_labels, k=5)
```

- En el análisis (analysis.py) se prueban varios valores: 1, 3, 5, 7, 9, 11, 15 y 21.

4.2 Número de colores en K-Means

En K-Means, la K representa cuántos grupos de color se quieren encontrar. El problema es que no todas las imágenes tienen el mismo número de colores importantes.

Para resolverlo, se utiliza una búsqueda automática de la mejor K. El método va probando distintos valores y observa cuánto mejora la distancia intra-clase. Cuando la mejora ya no es suficiente, se queda con el valor anterior.

En el código se usa una mejora mínima del 20%. Esto evita elegir demasiados colores y ayuda a quedarse con los colores principales de la prenda.

- En Kmeans.py, el valor por defecto es $K = 3$.
- En app.py, para detectar colores se empieza con $K = 2$ y luego se llama a `find_bestK(max_K=6)`, es decir, se busca automáticamente la mejor K hasta 6.
- En analysis.py, se compara también K fijo con 2, 3, 4 y 5.

4.3 Inicialización de centroides

K-Means depende de los centroides iniciales. Si se eligen mal, el resultado puede ser peor o tardar más en converger.

- first: usa los primeros píxeles como centroides. Es rápido, pero puede ser poco representativo.
- random: elige píxeles aleatorios. Puede funcionar bien, pero no siempre da el mismo resultado.
- custom: se parece a k-means++, intentando separar mejor los centroides iniciales.

Una buena inicialización puede reducir iteraciones y mejorar la calidad de los colores encontrados.

4.4 Características usadas en KNN

Para KNN se usa una representación muy simple: la imagen en escala de grises y aplanada en un vector. Esto hace que el sistema sea fácil de implementar y entender.

La parte negativa es que comparar píxel a píxel no siempre reconoce bien formas complejas. Si dos prendas están en posiciones distintas, el algoritmo puede confundirse. Como mejora, se podrían usar características más avanzadas, como bordes, histogramas o descriptores de forma.

5. Análisis de eficiencia

El análisis de eficiencia sirve para entender cuánto tarda el sistema y qué recursos necesita. Aunque el dataset del proyecto no es muy grande, es importante saber qué pasaría si aumentara el número de imágenes.

5.1 Eficiencia de KNN

KNN es un algoritmo muy sencillo porque prácticamente no entrena. Simplemente guarda las imágenes de entrenamiento.

- Entrenamiento: rápido, ya que solo almacena los datos.
- Predicción: más costosa, porque cada imagen nueva se compara con todas las imágenes de entrenamiento.
- Memoria: puede ser alta, porque hay que guardar todos los ejemplos.

Su coste aproximado en predicción es $N \cdot D$, donde N es el número de imágenes de entrenamiento y D el número de características. En este caso D es 4800 píxeles.

5.2 Eficiencia de K-Means

K-Means trabaja con los píxeles de cada imagen. En una imagen de 60 x 80 hay unos 4800 píxeles, así que el coste es razonable.

- Depende del número de píxeles.
- Depende del número de clusters K .
- Depende del número de iteraciones hasta que los centroides dejan de cambiar.

Su coste aproximado es $N \cdot K \cdot I$, donde N son los píxeles, K los grupos e I las iteraciones. Como K suele estar entre 2 y 6, el sistema es bastante rápido para imágenes pequeñas.

5.3 Eficiencia de find_bestK

El método find_bestK ejecuta K-Means varias veces con diferentes valores de K . Esto aumenta el tiempo de cálculo, pero tiene la ventaja de elegir automáticamente un número de colores más adecuado.

Como el máximo de K es pequeño, el coste extra es aceptable. Aun así, si el dataset fuera mucho más grande, habría que optimizar esta parte.

5.4 Resumen de eficiencia

Parte	Ventaja	Limitación
KNN	No necesita entrenamiento complejo.	La predicción puede ser lenta con muchos datos.
K-Means	Funciona bien con imágenes pequeñas.	Depende de K y de las iteraciones.
find_bestK	Elige mejor el número de colores.	Ejecuta K-Means varias veces.

6. Conclusiones

El proyecto demuestra que con algoritmos simples se puede construir un sistema útil para etiquetar imágenes de ropa. KNN permite reconocer la forma de la prenda y K-Means ayuda a detectar los colores principales.

- KNN funciona bien porque las imágenes están preparadas y tienen un formato parecido.
- K-Means es útil para resumir una imagen en pocos colores dominantes.
- La elección de K es importante tanto en KNN como en K-Means.
- El sistema es eficiente para datasets pequeños, pero podría volverse lento si aumentan mucho las imágenes.

Como mejora futura, se podrían usar características más avanzadas y métodos más rápidos de búsqueda de centroides.

En conclusión, el sistema cumple el objetivo principal: asignar etiquetas de forma y color a imágenes de productos para poder hacer búsquedas más inteligentes en una tienda online.